G8_Task

Hello,

Please read the following carefully:

As a Software Engineering research team at the Polytechnique Montréal (Canada), we are interested in understanding developers' assessment of coding task difficulty. You will be provided with a series of 12 coding questions. Questions are divided into two parts:

- First, you will be given three instructions representing coding tasks and you will be asked to write, in a few words, what is the underlying task represented by all these instructions. The instructions represent the same tasks, but they vary in wording and information they give. In a nutshell, you have to distill the essence of the task and write it in a few words.

- In a second time, you will be asked to assess how difficult implementing this ***task*** would be. Note we ask the assessment based on the task you will have worded in the previous part, not based on any of the instructions presented earlier. First, you will be asked your subjective judgment on a scale from Very Easy to Very Difficult. Then, to make judging the difficulty more relatable, we adopt a similar approach as what is used in agile development and planning poker, i.e. you will be asked to evaluate how long it would take to implement a task (in minutes here). Consider in your assessment the knowledge needed to implement the task, the external information you would need to use, whether you would need the assistance of a LLM such as ChatGPT (and so the time it would take to debug the obtained code) etc. Finally, you will have to explain in a few words why you gave such difficulty ratings.

To give you an idea of the range of tasks you will need to assess, the first section of this questionnaire will be two examples we devised to illustrate the questionnaire.

Completing the task will take about 30 minutes.

Note also that all examples are written using Python 3.10 syntax.

Please make sure to answer all the questions to the best of your ability.

Please note that your identity and personal data will not be disclosed, while we plan to use the aggregated results and anonymized responses as part of a scientific publication. If you have any questions about the questionnaire or our research, please do not hesitate to contact us.

By proceeding, you consent to take part in the study under the conditions described previously.

Thank you,

Florian Tambon (florian-2.tambon@polymtl.ca)
Cyrine Zid (cyrine.zid@polymtl.ca)
Amin Nikanjam (amin.nikanjam@polymtl.ca)
Foutse Khomh (foutse.khomh@polymtl.ca)
Giuliano Antoniol (giuliano.antoniol@polymtl.ca)

* Indique une question obligatoire

1. **Please enter your prolific ID :** *

Example of tasks

As we ask of you some estimation of difficulty and the tasks dealt in this questionnaire can be a bit different from what you can be used to as a developer, we give you examples of some tasks along with some assessments.

Note that this assessment is **our assessment**. You may have a different opinion based on your background. The idea of those examples is to give an extreme range of what type of tasks you can expect in the questionnaire.

As a reminder: You will first be given three instructions and asked to explain what is the task is about in a few words. Then, you will have to rate the difficulty of the ***task*** you just described (not accounting for particular instruction). Rating the difficulty is done in two steps: subjective difficulty and approximate time needed. Finally, you will be asked to say why you gave such rating.

Example 1

Instruction 1

```
def subtract(c1, c2):
        """
        Deliver the result of subtracting two complex numbers,
        specified as 'c1' and 'c2', as a complex output.
        :param c1: The first complex number,complex.
        :param c2: The second complex number,complex.
        :return: The difference of the two complex numbers,complex.
        """
```

Instruction 2

```python
def subtract(c1, c2):
        """
        The method receives two complex numbers 'c1' and 'c2', from which it
        subtracts 'c2.real' from 'c1.real' to determine the real portion, and
        subtracts 'c2.imag' from 'c1.imag' to determine the imaginary portion.
        It constructs and returns a new complex number using these results
        with the 'complex()' function.
        :param c1: The first complex number,complex.
        :param c2: The second complex number,complex.
        :return: The difference of the two complex numbers,complex.
        """
```

Instruction 3

```python
def subtract(c1, c2):
        """
        Subtracts two complex numbers.
        :param c1: The first complex number,complex.
        :param c2: The second complex number,complex.
        :return: The difference of the two complex numbers,complex.
        >>> complexCalculator = ComplexCalculator()
        >>> complexCalculator.subtract(1+2j, 3+4j)
        (-2-2j)
        """
```

Answer

The task encapsulated by the **subtract**

function

is to compute the subtraction between two complex numbers.

This task has a subjective difficulty of **Very Easy** as assessed by us and would take about **1 min**
to implement. There is no particular difficulty, the task is very
straightforward without any corner case, it just requires to know what a
complex is, and the standard Python library for it.

Example 2

Context

```python
import numpy as np
class MetricsCalculator2:
    """
    The class provides to calculate Mean Reciprocal Rank (MRR) and Mean Average Precision (MAP)
    based on input data, where MRR measures the ranking quality and MAP measures the average precision.
    """

    def __init__(self):
        pass

    @staticmethod
    def map(data):
        pass
```

Instruction 1

```python
def mrr(data):
        """
        Compute the MRR of the input data. MRR is a widely used evaluation index.
        It is the mean of reciprocal rank.
        :param data: the data must be a tuple, list 0,1,eg.([1,0,...],5).
        In each tuple (actual result,ground truth num),ground truth num is the total ground num.
        ([1,0,...],5), or list of tuple eg. [([1,0,1,...],5),([1,0,...],6),([0,0,...],5)].
        1 stands for a correct answer, 0 stands for a wrong answer.
        :return: if input data is list, return the recall of this list. if the input data is list of list, return the
        average recall on all list. The second return value is a list of precision for each input.
        >>> MetricsCalculator2.mrr(([1, 0, 1, 0], 4))
        >>> MetricsCalculator2.mrr([([1, 0, 1, 0], 4), ([0, 1, 0, 1], 4)])
        1.0, [1.0]
        0.75, [1.0, 0.5]

        """
```

Instruction 2

```python
def mrr(data):
        """
        Assess the Mean Reciprocal Rank (MRR) from the input 'data'.
        MRR assesses the average of reciprocal ranks of outcomes.
        The input structure 'data' must be a tuple consisting of a list of
        binary values and its total count or a list of such tuples.
        Each binary value (0 or 1) represents the correctness of an answer.
        If 'data' is a tuple, the function returns its mean reciprocal rank.
        If 'data' is a list, the function returns the overall average MRR along
        with a list showing reciprocal ranks for each component tuple.

        :param data: the data must be a tuple, list 0,1,eg.([1,0,...],5).
        In each tuple (actual result,ground truth num),ground truth num is the total ground num.
        ([1,0,...],5), or list of tuple eg. [([1,0,1,...],5),([1,0,...],6),([0,0,...],5)].
        1 stands for a correct answer, 0 stands for a wrong answer.
        :return: if input data is list, return the recall of this list. if the input data is list of list, return the
        average recall on all list. The second return value is a list of precision for each input.

        """
```

Instruction 3

```
def mrr(data):
        """
        Evaluate the Mean Reciprocal Rank (MRR) from 'data'. This metric calculates the mean of reciprocal
        ranks for correct answers. Input 'data' could be either a tuple containing binary correctness indicators
        and the aggregate count of answers, or multiple tuples of this type in a list. For each tuple, the function
        measures reciprocal ranks by multiplying the binary indicators by reciprocals of their positions, finding the
        earliest nonzero value to establish the rank of the first correct response. If the input 'data' is a tuple,
        the MRR calculated for this specific tuple is returned; if it's a list, the function summarizes by providing
        both the average MRR and a list of individual MRR values for each tuple.

        :param data: the data must be a tuple, list 0,1,eg.([1,0,...],5).
        In each tuple (actual result,ground truth num),ground truth num is the total ground num.
        ([1,0,...],5), or list of tuple eg. [([1,0,1,...],5),([1,0,...],6),([0,0,...],5)].
        1 stands for a correct answer, 0 stands for a wrong answer.
        :return: if input data is list, return the recall of this list. if the input data is list of list, return the
        average recall on all list. The second return value is a list of precision for each input.

        """
```

Answer

The task encapsulated by the **mrr** function is to compute the
Mean Reciprocal Rank of binary tuples data.

This task has a subjective difficulty of **Very Hard** as assessed by us and would take about
40 min
to implement. The difficulty mainly branches from dealing with the
tuple data and how the formula is processed accordingly (with a few corner cases). This can take
additional time if the Mean Reciprocal Rank is not known.

Question 1:

Below are instructions for implementing the function.

Instruction 1

```
def total_match(lst1, lst2):
    """
    Define a function 'total_match' that accepts two arguments, both lists of strings.
    It sums the characters in each list and returns the one with fewer characters.
    If the counts are the same, the first list is returned. Each list is iterated
    to compute the total character count.
    """
```

Instruction 2

```python
def total_match(lst1, lst2):
    """
    Write a function that accepts two lists of strings and returns the list that has
    total number of chars in the all strings of the list less than the other list.

    if the two lists have the same number of chars, return the first list.

    Examples
    total_match([], []) → []
    total_match(['hi', 'admin'], ['hI', 'Hi']) → ['hI', 'Hi']
    total_match(['hi', 'admin'], ['hi', 'hi', 'admin', 'project']) → ['hi', 'admin']
    total_match(['hi', 'admin'], ['hI', 'hi', 'hi']) → ['hI', 'hi', 'hi']
    total_match(['4'], ['1', '2', '3', '4', '5']) → ['4']
    """
```

Instruction 3

```python
def total_match(lst1, lst2):
    """
    Implement a function titled 'total_match' that receives two lists comprised
    of strings and delivers the list with the minimum aggregate characters,
    or the initial list if both are equal.
    """
```

2. In a few words, what is the task this function aims to implement? *

3. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

4. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

5. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 2:

Below are instructions for implementing the function:

Context

```python
class BitStatusUtil:
    """
    This is a utility class that provides methods for manipulating
    and checking status using bitwise operations.
    """

    def add(states, stat):
        pass

    @staticmethod
    def remove(states, stat):
        pass

    @staticmethod
    def check(args):
        pass


    @staticmethod
    def has(states, stat):

```

Instruction 1

```python
def has(states, stat):
    """
    Assess if the parameter 'states' holds the 'stat' parameter,
    with a preliminary verification of parameter legality.
    Should the bitwise AND computation between 'states'
    and 'stat' equate to 'stat', return 'True'.
    Otherwise, if the outcome does not replicate 'stat'
    or an error occurs during verification, return 'False'.
    :param states: Current status,int.
    :param stat: Specified status,int.
    :return: True if the current status contains the
                specified status,otherwise False,bool.
    """
```

Instruction 2

```python
def has(states, stat):
    """
    Check if the current status contains the specified status,and check
    the parameters wheather they are legal.
    :param states: Current status,int.
    :param stat: Specified status,int.
    :return: True if the current status contains the specified status,
                                        otherwise False,bool.
    >>> bit_status_util = BitStatusUtil()
    >>> bit_status_util.has(6,2)
    True
    """
```

Instruction 3

```
def has(states, stat):
    """
    Verify if 'states' encompasses 'stat' by initially checking
    the legality of both parameters. If the result of the bitwise
    AND between 'states' and 'stat' equates to 'stat', the function
    should return 'True'. If not, or if the validation of the
    parameters fails, the function should return 'False'.
    :param states: Current status,int.
    :param stat: Specified status,int.
    :return: True if the current status contains the specified
                                status,otherwise False,bool.
    """
```

6. In a few words, what is the task this function aims to implement? *

7. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

8. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

9. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 3:

Below are instructions for implementing the function:

Instruction 1

```python
def smallest_change(arr):
    """
    Construct the function 'smallest_change', taking an input array 'arr' of integers.
    The function identifies the minimum replacements necessary to render 'arr'
    a palindrome. It reverses 'arr' to 'arr_reversed', starts a counter at zero
    ('cnt'), and checks the first half of 'arr' against 'arr_reversed'.
    Each differing value increments 'cnt'. After the loop, 'cnt' is returned,
    indicating the count of required changes.
    """
```

Instruction 2

```python
def smallest_change(arr):
    """
    Given an array arr of integers, find the minimum number of elements that
    need to be changed to make the array palindromic. A palindromic array is
    an array that is read the same backwards and forwards. In one change,
    you can change one element to any other element.

    For example:
    smallest_change([1,2,3,5,4,7,9,6]) == 4
    smallest_change([1, 2, 3, 4, 3, 2, 2]) == 1
    smallest_change([1, 2, 3, 2, 1]) == 0
    """
```

Instruction 3

```python
def smallest_change(arr):
    """
    Develop a function named 'smallest_change' which receives an array of
    integers and returns the least number of modifications required to make
    the array a palindrome. By comparing elements from opposite ends of the
    array and moving inwards, it should count the discrepancies between them.
    This total will be the count of changes required to make the sequence
    identical from both ends.
    """
```

10. In a few words, what is the task this function aims to implement? *

11. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

12. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

13. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

<u>Question 4:</u>

Below are instructions for implementing the function:

Context

```
class ShoppingCart:
    """
    The class manages items, their prices, quantities, and allows to for add,
    removie, view items, and calculate the total price.
    """

    def __init__(self):
        """
        Initialize the items representing the shopping list as an empty dictionary
        """
        self.items = {}


    def remove_item(self, item, quantity=1):
        pass

    def view_items(self) -> dict:
        pass

    def total_price(self) -> float:
        pass
```

Instruction 1

```
def add_item(self, item, price, quantity=1):
    """
    Insert or update 'item' details in the 'self.items' dictionary using
    parameters 'item', 'price', and 'quantity'. The parameter 'quantity'
    has a default value of 1. If 'item' is already in 'self.items',
    update its 'price' and 'quantity'. Otherwise, create a new entry
    with these details. This method keeps the shopping cart updated,
    does not return any results, and ensures no duplicate items are in
    the dictionary with inaccurate details.
    :param item: string, Item to be added
    :param price: float, The price of the item
    :param quantity:int, The number of items, defaults to 1
    :return:None"

    """
```

Instruction 2

```
def add_item(self, item, price, quantity=1):
    """
    Add item information to the shopping list items, including price
    and quantity. The default quantity is 1
    :param item: string, Item to be added
    :param price: float, The price of the item
    :param quantity:int, The number of items, defaults to 1
    :return:None
    >>> shoppingcart = ShoppingCart()
    >>> shoppingcart.add_item("apple", 1, 5)
    self.items = {"apple":{"price":1, "quantity":5}}
    """
```

Instruction 3

```
def add_item(self, item, price, quantity=1):
    """
    Incorporate data about 'item' into the dictionary 'self.items',
    setting 'quantity' to 1 by default if it's not given. There is
    no return value for this function.
    :param item: string, Item to be added
    :param price: float, The price of the item
    :param quantity:int, The number of items, defaults to 1
    :return:None
    """
```

14. In a few words, what is the task this function aims to implement? *

15. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

16. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

17. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

<u>Question 5:</u>

Below are instructions for implementing the function:

Instruction 1

```python
def triples_sum_to_zero(l: list):
    """
    Implement a function named 'triples_sum_to_zero' that takes
    in a list of integers. The purpose of the function is to
    find any three separate integers in the list whose sum is zero.
    Using three nested loops to pick each integer, it confirms that
    no two of these integers have the same index, and that their sum
    equals zero. The function returns True upon discovery of such a
    triplet, or False if no triplet meets these criteria after every
    possible iteration.
    """
```

Instruction 2

```python
def triples_sum_to_zero(l: list):
    """
    triples_sum_to_zero takes a list of integers as an input.
    it returns True if there are three distinct elements in
    the list that sum to zero, and False otherwise.

    >>> triples_sum_to_zero([1, 3, 5, 0])
    False
    >>> triples_sum_to_zero([1, 3, -2, 1])
    True
    >>> triples_sum_to_zero([1, 2, 3, 7])
    False
    >>> triples_sum_to_zero([2, 4, -5, 3, 9, 7])
    True
    >>> triples_sum_to_zero([1])
    False
    """
```

Instruction 3

```python
def triples_sum_to_zero(l: list):
    """
    Develop a function 'triples_sum_to_zero' which is passed a list of integers.
    The function aims to identify any set of three distinct integers from the
    list that sum to zero. This is done by using three nested loops to select
    the integers, making sure their indices are not the same, and that the
    sum of these integers equals zero. The function will return True if it finds
    such a combination, and False otherwise after completing all iterations.
    """
```

18. In a few words, what is the task this function aims to implement? *

19. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

20. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

21. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 6:

Below are instructions for implementing the function:

Context

```python
class EncryptionUtils:
    """
    This is a class that provides methods for encryption,
    including the Caesar cipher, Vigenere cipher, and Rail
    Fence cipher.
    """

    def __init__(self, key):
        """
        Initializes the class with a key.
        :param key: The key to use for encryption, str.
        """
        self.key = key

    def caesar_cipher(self, plaintext, shift):
        pass

    def vigenere_cipher(self, plaintext):
        pass
```

Instruction 1

```python
def rail_fence_cipher(self,plain_text, rails):
    """
    Encrypt the provided text 'plaintext' by employing
    the Rail Fence cipher method and specifying the number of
    rails as 'rails'. Deliver the encrypted text. A 'fence',
    which consists of multiple sublists each representing a
    rail with placeholders, is initialized first. An internal
    'direction' mechanism switches for upward and downward
    movement across the rails. Characters are sequentially
    inserted from the first rail downwards and direction is
    reversed upon reaching any extreme. Lastly, the encrypted
    output is formed by aggregating characters from all rows in
    'fence' excluding placeholders.
    :param plaintext: The plaintext to encrypt, str.
    :return: The ciphertext, str.
    """
```

Instruction 2

```
def rail_fence_cipher(self,plain_text, rails):
    """
    Encrypts the plaintext using the Rail Fence cipher.
    :param plaintext: The plaintext to encrypt, str.
    :return: The ciphertext, str.
    >>> e = EncryptionUtils("key")
    >>> e.rail_fence_cipher("abc", 2)
    'acb'
    """
```

Instruction 3

```
def rail_fence_cipher(self,plain_text, rails):
    """
    Using the Rail Fence cipher method, encrypt the message 'plaintext'
    with 'rails' number of levels and return the encoded text as a string.
    :param plaintext: The plaintext to encrypt, str.
    :return: The ciphertext, str.
    """
```

22. In a few words, what is the task this function aims to implement? *

23. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

24. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

25. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

<u>Question 7:</u>

Below are instructions for implementing the function:

Instruction 1

```python
def prime_length(string):
    """
    Construct a function 'prime_length' which accepts a single parameter,
    a string labeled 'string', and determines if its length is a prime
    number. It returns 'True' for a prime length and 'False' otherwise.
    Inside it, there’s a nested function 'is_prime' that checks the
    primality of a number 'a', thus 'is_prime' returns 'True' if 'a'
    is a prime number and 'False' otherwise. It evaluates if 'a'
    is divisible by any number from 2 to the integer square root of
    'a' plus one without remainders. The top function then calculates
    the string length, checking it using 'is_prime'.
    """
```

Instruction 2

```python
def prime_length(string):
    """
    Write a function that takes a string and returns
    True if the string length is a prime number or
    False otherwise

    Examples
    prime_length('Hello') == True
    prime_length('abcdcba') == True
    prime_length('kittens') == True
    prime_length('orange') == False
    """
```

Instruction 3

```python
def prime_length(string):
    """
    Implement a function named 'prime_length' that takes one
    string parameter and assesses if its length is a prime number.
    It returns true for primes and false otherwise. An enclosed
    function aids this by examining if any number from 2 up to
    the integer's square root divides the length evenly.
    """
```

26. In a few words, what is the task this function aims to implement? *

27. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

28. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

29. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 8:

Below are instructions for implementing the function:

Context

```python
import math
class AreaCalculator:
    """
    This is a class for calculating the area of different shapes, 
    including circle, sphere, cylinder, sector and annulus.
    """

    def __init__(self, radius):
        """
        Initialize the radius for shapes.
        :param radius: float
        """
        self.radius = radius

    def calculate_circle_area(self):
        pass

    def calculate_sphere_area(self):
        pass

    def calculate_cylinder_area(self, height):
        pass

    def calculate_sector_area(self, angle):
        pass



```

Instruction 1

```python
def calculate_annulus_area(self, inner_radius, outer_radius):
    """
    calculate the area of annulus based on inner_radius 
    and out_radius
    :param inner_radius: inner radius of sector, float
    :param outer_radius: outer radius of sector, float
    :return: area of annulus, float
    >>> areaCalculator.calculate_annulus_area(2, 3)
    15.707963267948966
    """
```

Instruction 2

```python
def calculate_annulus_area(self, inner_radius, outer_radius):
    """
    Using the 'inner_radius' and 'outer_radius', calculate
    the area of an annulus and ensure the result is returned
    as a float.
    :param inner_radius: inner radius of sector, float
    :param outer_radius: outer radius of sector, float
    :return: area of annulus, float
    """
```

Instruction 3

```python
def calculate_annulus_area(self, inner_radius, outer_radius):
    """
    Compute the area of an annulus using the values 'inner_radius'
    and 'outer_radius' and return the resulting area as a float type.
    The function should utilize the formula
    `math.pi * (outer_radius ** 2 - inner_radius ** 2)`,
    where it first calculates the area of the larger circle using
    the square of 'outer_radius', multiplies this by `math.pi`,
    and subtracts from it the area of the smaller circle calculated
    in a similar manner using 'inner_radius'.
    :param inner_radius: inner radius of sector, float
    :param outer_radius: outer radius of sector, float
    :return: area of annulus, float
    """
```

30. In a few words, what is the task this function aims to implement? *

____

31. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- [ ] Very Easy
- [ ] Easy
- [ ] Not Easy Nor Difficult
- [ ] Difficult
- [ ] Very Difficult

32. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

33. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

<u>Question 9:</u>

Below are instructions for implementing the function:

Instruction 1

```python
def how_many_times(string: str, substring: str) -> int:
    """
    Implement a function named 'how_many_times' that takes
    a string and a substring as inputs. Its objective is to
    count and return the number of times the given substring
    appears in the string, covering overlapping situations.
    Starting with a count of zero, the function moves through
    the string by looping. At every step of the loop, it checks
    if the substring starts at the new segment of the string
    from the present loop index. When the substring is identified
    at this start, the counter is increased. Following the loop,
    the function issues the accumulated count of occurrences.
    """
```

Instruction 2

```python
def how_many_times(string: str, substring: str) -> int:
    """
    Find how many times a given substring can be found in
    the original string. Count overlaping cases.
    >>> how_many_times('', 'a')
    0
    >>> how_many_times('aaa', 'a')
    3
    >>> how_many_times('aaaa', 'aa')
    3
    """
```

Instruction 3

```python
def how_many_times(string: str, substring: str) -> int:
    """
    Write a function named 'how_many_times' which counts
    and returns the number of times a specified substring
    appears in a given string, including overlapping
    occurrences.
    """
```

34. In a few words, what is the task this function aims to implement? *

35. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

36. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

37. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 10:

Below are instructions for implementing the function:

Context

```python
class ClassRegistrationSystem:
    """
    This is a class as a class registration system,
    allowing to register students, register them for classes,
    retrieve students by major, get a list of all majors,
    and determine the most popular class within a specific major.
    """

    def __init__(self):
        """
        Initialize the registration system with the attribute
        students and students_registration_class.
        students is a list of student dictionaries,
        each student dictionary has the key of name and major.
        students_registration_class is a dictionaries,
        key is the student name, value is a list of class names
        """
        self.students = []
        self.students_registration_classes = {}

    def register_student(self, student):
        pass

    def register_class(self, student_name, class_name):
        pass

    def get_students_by_major(self, major):
        pass

    def get_most_popular_class_in_major(self, major):
        pass
```

Instruction 1

```python
def get_all_major(self):
    """
    get all majors in the system
    :return a list of majors
    >>> registration_system = ClassRegistrationSystem()
    >>> registration_system.students = [{"name": "John",
                        "major": "Computer Science"}],
    >>> registration_system.get_all_major(student1)
    ["Computer Science"]

    """
```

Instruction 2

```python
def get_all_major(self):
    """
    Produce and return a list of all distinct majors.
    Iterate over each student within the student list,
    and for each one, check if their 'major' from their
    dictionary is already in an accumulative list. If not,
    include this new major. Eventually, return the list
    which now contains only unique majors.
    :return a list of majors
    """
```

Instruction 3

```python
def get_all_major(self):
    """
    Extract each unique major from the 'students' list,
    returning a list containing these unique majors.
    In the 'get_all_major' function, it traverses every
    student dictionary within the 'students' list and
    adds the student's 'major' to a local variable
    'major_list' if it is not already there. Once the
    process is complete, the unique majors collected in
    'major_list' are returned.
    :return a list of majors
    """
```

38. In a few words, what is the task this function aims to implement? *

39. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

40. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

41. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

<u>Question 11:</u>

Below are instructions for implementing the function

Instruction 1

```python
from typing import List, Optional

def longest(strings: List[str]) -> Optional[str]:
    """
    Write a function 'longest' receiving a string list called 'strings'.
    It's tasked with selecting and returning the initially encountered
    longest string from this list. If 'strings' contains no elements,
    it yields 'None'. When equal maximum lengths are found in several
    strings, it will return the first found string. The initial step
    of the function checks for an empty 'strings' list, yielding 'None'
    if empty. Subsequently, it calculates the greatest string length
    by iterating with a generator expression through each string 'x'
    in 'strings', computing 'len(x)', and storing this in 'maxlen'.
    The function then loops over 'strings' to find and return the
    earliest string 's' that matches 'maxlen' in length.
    """
```

Instruction 2

```python
from typing import List, Optional

def longest(strings: List[str]) -> Optional[str]:
    """
    Out of list of strings, return the longest one.
    Return the first one in case of multiple
    strings of the same length. Return None in case
    the input list is empty.
    >>> longest([])

    >>> longest(['a', 'b', 'c'])
    'a'
    >>> longest(['a', 'bb', 'ccc'])
    'ccc'
    """
```

Instruction 3

```python
from typing import List, Optional

def longest(strings: List[str]) -> Optional[str]:
    """
    Implement a function titled 'longest' which accepts
    a list 'strings' of strings. The function aims to
    retrieve and return the initial longest string present
    in 'strings'. If there are no elements in 'strings',
    it will return 'None'. If there exists multiple strings
    of the same maximum length, it will return the first
    one encountered. At the beginning, the function checks
    if 'strings' is empty and returns 'None' if it is. Next,
    by utilizing a generator expression that applies 'len(x)'
    to each string 'x' in 'strings', it calculates and stores
    the largest string length in 'maxlen'. Then, it proceeds
    to iterate over 'strings' to return the first string 's'
    that has a length equal to 'maxlen'.
    """
```

42. In a few words, what is the task this function aims to implement? *

43. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

44. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

45. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Question 12:

Below are instructions for implementing the function:

Context

```python
import itertools
class ArrangementCalculator:
    """
    The Arrangement class provides permutation calculations
    and selection operations for a given set of data elements.
    """

    def __init__(self, datas):
        """
        Initializes the ArrangementCalculator object with a list of datas.
        :param datas: List, the data elements to be used for arrangements.
        """
        self.datas = datas

    def count(n, m=None):
        pass

    def select(self, m=None):
        pass

    def select_all(self):
```

Instruction 1

```python
def count_all(n):
    """
    This function determines the overall total of
    all viable configurations by selecting from at
    least one item to all 'n' items from a set of
    'n' items. For each possible number of selections
    from 1 to 'n', the function computes the
    arrangements and adds these numbers together.
    It uses a pre-established method for determining
    arrangements for each number of selections and
    sums the results.
    :param n: int, the total number of items.
    :return: int, the count of all arrangements.
    """
```

Instruction 2

```python
def count_all(n):
    """
    Counts the total number of all possible arrangements
    by choosing at least 1 item and at most n items from
    n items.
    :param n: int, the total number of items.
    :return: int, the count of all arrangements.
    >>> ArrangementCalculator.count_all(4)
    64
    """
```

Instruction 3

```python
def count_all(n):
    """
    Accumulates and returns the sum of all feasible
    arrangements for choosing from one to 'n'
    items out of 'n' total items.
    :param n: int, the total number of items.
    :return: int, the count of all arrangements.
    """
```

46. In a few words, what is the task this function aims to implement? *

47. How would you characterize the subjective difficulty of the task? *

Une seule réponse possible.

- Very Easy
- Easy
- Not Easy Nor Difficult
- Difficult
- Very Difficult

48. How would you rate the difficulty of this task, in minutes it would take to implement? *

Une seule réponse possible.

- 1 min
- 5 min
- 10 min
- 20 min
- 40 min
- > 40 min

49. Why did you consider this task of such difficulty? Explain your reasoning (time it would take to search a concept, difficulty of the programming structure involved, etc.) *

Demographics Information

50. What is your highest education qualification? *

Une seule réponse possible.

- Ph.D.
- Master's Degree
- Bachelor Degree
- Autre :

51. What is your field of work? (Engineer, Researcher, Developer, etc.) *

52. How many years of development experience do you have? *

Une seule réponse possible.

- Less than 5
- Between 5 and 10
- More than 10

53. Thank you for completing the survey! Any feedback you would like to provide?